

Beyond Compilation: Functional Correctness and Static-Analysis Compliance in LLM-Generated TypeScript

Yehezkiel Gunawan
Bina Nusantara University
Jakarta, Indonesia
yehezkiel.gunawan@binus.ac.id

Abstract—An LLM-generated TypeScript program may fail strict compilation, compile but violate a behavioral requirement, or pass its tests while violating selected project-level static-analysis rules. We examined these outcomes separately using 15 deterministic TypeScript utility tasks, three models, and 45 generated programs. Each program underwent source extraction, safety inspection, and strict TypeScript compilation; compilable programs then proceeded to task-specific Vitest tests and ESLint analysis. Of the 45 programs, 37 compiled, 36 passed all visible tests, and 34 passed the complete evaluation gate. All eight compilation failures involved possibly undefined indexed access under strict TypeScript settings. Testing rejected one additional compilable program, and ESLint identified violations in two programs that had passed every visible test. In the evaluated tasks and models, compilation, functional correctness, and compliance with selected static-analysis rules were distinct outcomes. Compilation was a useful first filter, but it did not replace behavioral testing or project-specific static analysis.

Index Terms—large language models, code generation, TypeScript, functional correctness, static analysis, empirical software engineering

I. INTRODUCTION

A TypeScript program generated by a large language model (LLM) may fail strict compilation, compile but violate a behavioral requirement, or pass its tests while violating project-level static-analysis rules. These outcomes form three sequential but non-equivalent gates: compilation, functional tests, and selected static analysis. TypeScript makes the first distinction especially visible. Code that appears executable as JavaScript can still fail a strict type contract when indexed values, optional properties, or generic types retain the possibility of undefined.

Execution-based benchmarks such as HumanEval made tests central to code-generation evaluation [1]. Later studies examined variation across languages and models, properties beyond functional correctness, and tasks extending from function synthesis to repository issue resolution [2]–[6]. Comparatively less attention has been given to compact, reproducible evaluations of strict TypeScript utilities that report compilation, behavioral, and static-analysis outcomes separately. This study addresses that narrower gap as an incremental empirical contribution.

The benchmark uses 15 deterministic utilities that are independently testable and small enough for source-level in-

spection. They represent common TypeScript development concerns while excluding frameworks, databases, browsers, and external services that would add competing explanations for failure.

This study asks how reliably contemporary LLMs produce TypeScript that survives three conventional development checks. We address the following questions:

RQ1: To what extent does generated TypeScript compile under strict compiler settings?

RQ2: To what extent are compilable programs functionally correct under task-specific tests?

RQ3: What violations of selected static-analysis rules remain, including in programs that pass all tests?

The contribution is a reproducible TypeScript corpus, a controlled 45-generation comparison under one prompt, and an empirical account of attrition across the three gates. The artifact, including tasks, prompts, raw generations, evaluation records, and report scripts, is available online.¹

II. RELATED WORK

A. Functional correctness

HumanEval established execution against unit tests as a standard measure for function synthesis [1]. Xu et al. found substantial variation across code models and programming languages [2], while Nguyen and Nadi evaluated Copilot suggestions with tests and complexity measures [3]. The test oracle determines how much this evidence establishes: EvalPlus exposed programs accepted by the original HumanEval tests after expanding the suites [7]. Our tests therefore measure conformance to specified cases rather than proof of correctness.

B. Properties beyond functional correctness

Passing tests does not establish every relevant property of generated code. Yetiştiren et al. assessed several quality dimensions [4], Al Madi examined readability and complexity [8], and Pearce et al. evaluated security [9]. Our ESLint stage has a narrower construct: compliance with selected project rules, not maintainability, security, or production readiness.

¹<https://github.com/yehezkielgunawan/icdees-2026>

C. Robustness and nondeterminism

Generation conditions affect reproducibility. Semantically similar prompts can change Copilot outputs [10]; prompt language and repeated runs can alter suggestion outcomes [11]. Ouyang et al. report nondeterminism in ChatGPT code generation [12]. We therefore fixed one prompt and retained the generated artifacts. This control removes prompt strategy as an independent variable, but one generation per model-task pair cannot characterize output variability.

D. Software-engineering task scope

Evaluation now spans isolated functions, class-level generation, repository issue resolution, and agents that edit repositories and run development commands [1], [5], [6], [13]. Our smaller utilities trade application-level realism for deterministic execution, manual inspectability, and isolation of strict TypeScript behavior. They provide ecosystem-specific evidence rather than another general-purpose benchmark, consistent with the range of evaluation practices catalogued by Hou et al. [14].

III. METHOD

A. Research design and tasks

The independent variable was the model. Dependent variables were strict compilation, full test-suite success, selected ESLint findings, and passage through all gates. The task, prompt, execution order, test suite, compiler, lint rules, and dependency lock were held constant across models.

The corpus contains 15 tasks divided evenly among five categories (Table I). Selection required deterministic behavior, independent testability, bounded source size, manual inspectability, and relevance to TypeScript or web-backend development. Examples include building a category tree, validating pagination parameters, normalizing URL paths, limiting asynchronous concurrency, and implementing cursor pagination. We excluded React, Next.js, databases, browsers, and external APIs so that framework behavior, environment state, and remote services would not confound the three measured outcomes. Each task has a required exported signature, a reference implementation, and four to six Vitest cases. The corpus contains 75 tests in total. Reference implementations passed all tests, strict compilation, and ESLint under the same evaluator.

TABLE I
TYPESCRIPT TASK SET

Category	No.	Representative task
Data transformation	3	Group records by a derived key
Validation	3	Validate a registration form
String and URL processing	3	Convert a title to a URL-safe slug
Async and Promise utilities	3	Retry an asynchronous operation
Web backend utilities	3	Implement an in-memory TTL cache

B. Generation procedure

We evaluated three identifiers from the OpenAI provider: `gpt-5.6-sol`, `gpt-5.6-terra`, and `gpt-5.6-luna`, labeled Sol, Terra, and Luna. These identifiers were active in the recorded provider snapshot but do not identify immutable public model revisions. Each model generated one program for each task, for 45 generations. The campaign ran on September 3, 2026 through OpenCode 1.18.26.

One prompt template required source code only, the exact signature, no imports or external dependencies, no `any`, no TypeScript suppression comments, and no process, file-system, network, or global mutable-state access. Holding this template constant prevented prompt strategy from becoming another independent variable. The benchmark agent could not call tools and was limited to one step. A temperature of zero was requested, but the runtime marked temperature unsupported for all three models and recorded no applied value. The configured 4096-token output limit was recorded but not transmitted by the generation call, so it is not treated as an enforced control. The campaign manifest froze the task inputs, prompt, study configuration, and dependency lockfile before generation, preventing post-hoc changes to those inputs. All raw responses, extracted sources, hashes, and metadata were retained. All 45 generations and extractions succeeded without normalization changes.

C. Evaluation pipeline

The evaluator first checked for empty output and disallowed operations. It then compiled each candidate and its tests with TypeScript 5.8.2 using `strict`, `noImplicitAny`, `noUncheckedIndexedAccess`, and `exactOptionalPropertyTypes`, among other module-isolation settings. Strict compilation tests whether generated TypeScript satisfies a stronger static contract than successful JavaScript execution. Compile failure stopped later stages. Compilable candidates proceeded to their Vitest 3.2.4 suite and ESLint 9.35.0 in parallel. Deterministic visible tests provided reproducible checks of specified behavior, while ESLint supplied a lightweight, reproducible check of selected project rules.

A candidate was *fully correct* when at least one test ran and every test passed. A candidate was *static-clean* when ESLint completed with zero errors. Selected rules included a cyclomatic-complexity maximum of 12, `prefer-const`, equality and control-flow rules, and TypeScript rules for explicit `any` and unused variables. The *full gate* required successful extraction, compilation, full correctness, and zero ESLint errors. Compilation, functional, and full-gate rates use all generations as their denominator. Static-clean and compiled-but-incorrect rates use compilable programs; correct-but-static-issue rates use fully correct programs. Given 15 paired tasks and one sample per cell, we report counts and percentages without hypothesis tests or model-ranking claims.

IV. RESULTS

Table II reports each model and the aggregate outcomes. Fig. 1 shows the same sequential gates to make attrition explicit.

A. RQ1: Strict compilation

Thirty-seven of 45 candidates compiled, an aggregate rate of 82.2%. The observed model rates ranged from 12/15 (80.0%) for Sol and Terra to 13/15 (86.7%) for Luna. Every rejected candidate had exactly one compiler error, and all eight errors involved possibly undefined indexed access under `noUncheckedIndexedAccess`.

The failures were concentrated in three tasks rather than spread across the corpus. Sol and Terra failed task 08 by constructing an array whose inferred element type remained `string | undefined`. All three models failed task 11 when passing `items[index]`, typed as `T | undefined`, to a worker expecting `T`. All three failed task 15 through a similar access to the last page item. These are strict type failures, not observed test failures: the pipeline did not run Vitest or ESLint after compilation failed.

B. RQ2: Functional correctness

Among 37 compilable candidates, 36 passed their complete task suite (97.3%). Relative to all generated candidates, the functional success rate was 36/45 (80.0%). Across compiled programs, 178 of 179 executed test cases passed, but candidate-level outcomes remain the unit of analysis.

The sole compiled-but-incorrect candidate was Terra’s registration validator in task 04. It passed four of five tests. For an empty object, it reported missing username, email, and password fields but omitted `confirmPassword`. Both password fields evaluated to `undefined`, so their strict equality comparison returned `true`. This case directly separates compilation from behavioral compliance, although its frequency, 1/37, does not support a claim that compiled programs often failed the tests.

C. RQ3: Static-analysis compliance and the full gate

Thirty-five of 37 compiled candidates were static-clean (94.6%). Both lint failures had passed all functional tests. Luna’s URL-path normalizer had cyclomatic complexity 13 against the configured maximum of 12. Terra’s Promise-timeout utility declared a reassignment-free timer with `let`, violating `prefer-const`. No linted candidate used explicit `any`, suppression comments, or generated a warning.

The static stage reduced the fully correct set from 36 to 34 full-gate programs. Sol and Luna each passed the complete gate for 12/15 candidates (80.0%); Terra passed for 10/15 (66.7%). These differences are descriptive observations from one run, not estimates of stable model ordering.

V. DISCUSSION

A. Compilation is useful but incomplete

Compilation rejected eight programs before behavioral evaluation. Every failure reflected the same TypeScript-specific

problem: runtime reasoning about bounds or existence did not narrow an indexed value enough to satisfy its declared type. Values remained `T | undefined` or `string | undefined` until the implementation supplied explicit narrowing or a justified non-null assertion. The generated logic could appear executable while failing TypeScript’s static proof requirements.

In this corpus, strict compilation eliminated more candidates than the subsequent visible test suites, which rejected one additional compilable program. This comparison identifies compilation as an effective first filter for these tasks, not as a generally stronger validation method. The registration defect shows why the distinction matters: a program can satisfy the type checker and still violate a boundary-case requirement.

B. Behavioral tests and static analysis answer different questions

Terra’s validator treated two missing password fields as equal because both evaluated to `undefined`. The program was type-correct, but a visible test exposed the omitted validation error. Test evidence remains finite: EvalPlus has shown that expanded suites can reject programs accepted by smaller suites [7]. Here, “fully correct” means success on four to six visible tests, not proof against the complete input space.

The two ESLint findings occurred in test-passing programs and did not indicate behavioral defects. One captured a reassignment-free `let`; the other marked complexity one point above the configured threshold. They show that test success does not imply compliance with selected project rules. They do not establish maintainability, security, or production readiness, which require broader measures and, in some cases, human assessment.

C. Conventional engineering gates remain useful

Generated TypeScript should be treated as candidate code and subjected to the same compiler, tests, and static-analysis checks as human-written code. Each gate answers a different question: strict compilation checks the type contract, tests exercise specified behavior, and ESLint enforces selected project rules. Explicit denominators are also necessary because compile failures never reach later stages. Reporting a static-clean rate over all generations would conflate rejection with unobserved lint behavior.

VI. SCOPE, LIMITATIONS, AND THREATS TO VALIDITY

This study covers 45 single-attempt generations across 15 isolated TypeScript utility tasks and three provider model identifiers. It measures strict compilation, performance on visible task-specific tests, and compliance with selected ESLint rules. The findings characterize this controlled campaign; they do not establish general LLM coding performance, production readiness, or stable differences among models.

Construct validity. Unit tests approximate functional correctness and cover only four to six cases per task. One task description included URL query and fragment handling that its tests did not exercise, which further limits the “fully correct”

TABLE II

RESULTS BY MODEL. COMPILATION, FULL CORRECTNESS, AND FULL GATE USE GENERATED CANDIDATES AS DENOMINATORS. STATIC-CLEAN AND COMPILED-INCORRECT USE COMPILED CANDIDATES; CORRECT-STATIC-ISSUE USES FULLY CORRECT CANDIDATES.

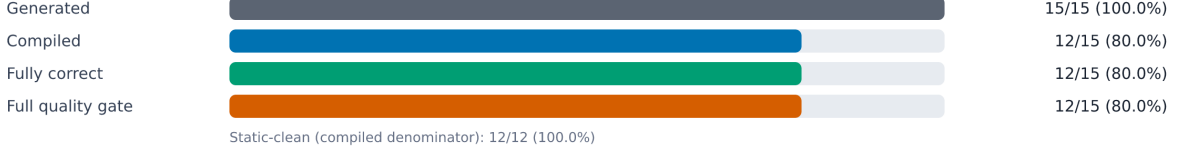
Model	Compiled	Fully correct	Static-clean	Full gate	Compiled, incorrect	Correct, static issue
GPT-5.6 Sol	12/15 (80.0%)	12/15 (80.0%)	12/12 (100.0%)	12/15 (80.0%)	0/12 (0.0%)	0/12 (0.0%)
GPT-5.6 Terra	12/15 (80.0%)	11/15 (73.3%)	11/12 (91.7%)	10/15 (66.7%)	1/12 (8.3%)	1/11 (9.1%)
GPT-5.6 Luna	13/15 (86.7%)	13/15 (86.7%)	12/13 (92.3%)	12/15 (80.0%)	0/13 (0.0%)	1/13 (7.7%)
All	37/45 (82.2%)	36/45 (80.0%)	35/37 (94.6%)	34/45 (75.6%)	1/37 (2.7%)	2/36 (5.6%)

Quality-gate survival by model

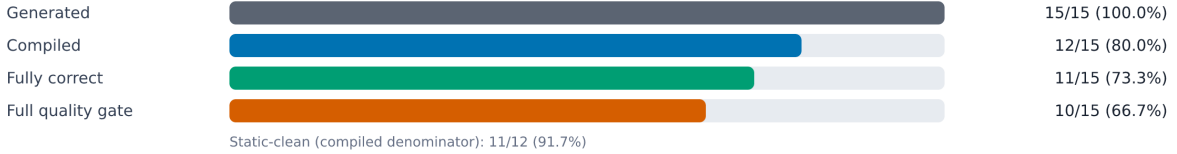
Evaluated generations: 45.

Count / denominator (percentage)

GPT-5.6 Sol



GPT-5.6 Terra



GPT-5.6 Luna

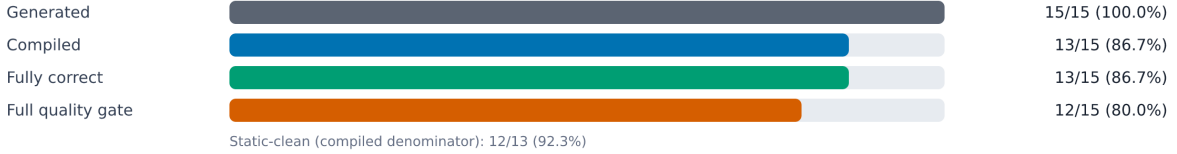


Fig. 1. Quality-gate survival for the 45 primary generations. Static-clean rates are shown separately because their denominator is the number of compilable candidates.

label. ESLint represents a small set of static rules, not general maintainability or security. Strict compilation failures mainly measure explicit handling of conservative indexed-access types and should not be interpreted as algorithmic errors.

Internal and conclusion validity. Each model-task pair has one generation. Because temperature control was unsupported, the study cannot separate stable model behavior from sampling variation. Fixed prompts and complete raw-output capture improve auditability but do not remove nondeterminism or provider updates [10], [12]. The small paired sample precludes strong ranking claims. The manifest froze tasks, prompt, study configuration, and a dependency lockfile, but did not bind the evaluator source revision or benchmark-agent configuration. The exact campaign-time Node version, operating system, and provider-side model revisions were not captured.

External validity. The corpus contains 15 isolated TypeScript utilities, three provider identifiers, one prompt, and visible tests. Results do not generalize to full applications, interactive assistants, repository agents, other languages, or other model revisions. The compact design improves control

and reproducibility but is less realistic than class-level or repository-level benchmarks.

VII. CONCLUSION

Among 45 generated TypeScript programs, strict compilation rejected eight, testing exposed one additional behavioral defect, and ESLint identified selected rule violations in two otherwise test-passing programs. These outcomes show that compilation, functional correctness, and static-analysis compliance were distinct gates for the evaluated tasks and models. Compilation was an effective first filter in this corpus, but neither compilation nor test success established passage through the complete evaluation pipeline. Generated TypeScript should therefore undergo ordinary project validation before integration.

ACKNOWLEDGMENT

The author used OpenCode with GPT-5.6-sol and GPT-5.6-luna primarily for technical coding tasks; support for the systematic literature review through Exa AI MCP, including source discovery, metadata verification, and source

organization; and language editing, including paraphrasing and correction of language errors. The author independently reviewed and validated all AI-assisted outputs, verified the empirical claims and cited sources, and assumes full responsibility for the accuracy, interpretation, and final content of the manuscript.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [2] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. ACM, 2022, pp. 1–10. [Online]. Available: <https://doi.org/10.1145/3520312.3534862>
- [3] N. Nguyen and S. Nadi, "An empirical evaluation of GitHub Copilot's code suggestions," in *Proceedings of the 19th International Conference on Mining Software Repositories*. ACM, 2022, pp. 1–5. [Online]. Available: <https://doi.org/10.1145/3524842.3528470>
- [4] B. Yetistiren, I. Ozsoy, and E. Tuzun, "Assessing the quality of GitHub Copilot's code generation," in *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering*. ACM, 2022, pp. 62–71. [Online]. Available: <https://doi.org/10.1145/3558489.3559072>
- [5] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Evaluating large language models in class-level code generation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, 2024, pp. 1–13. [Online]. Available: <https://doi.org/10.1145/3597503.3639219>
- [6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations*, 2024, pp. 54 107–54 157. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html
- [7] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 21 558–21 572. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html
- [8] N. Al Madi, "How readable is model-generated code? examining readability and visual inspection of GitHub Copilot," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2022, pp. 1–5. [Online]. Available: <https://doi.org/10.1145/3551349.3560438>
- [9] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *2022 IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 754–768. [Online]. Available: <https://doi.org/10.1109/SP46214.2022.9833571>
- [10] A. Mastropalo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, "On the robustness of code generation techniques: An empirical study on GitHub Copilot," in *2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, 2023, pp. 2149–2160. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00181>
- [11] K. Koyanagi, D. Wang, K. Noguchi, M. Kondo, A. Serebrenik, Y. Kamei, and N. Ubayashi, "Exploring the effect of multiple natural languages on code suggestion using GitHub Copilot," in *Proceedings of the 21st International Conference on Mining Software Repositories*. ACM, 2024, pp. 481–486. [Online]. Available: <https://doi.org/10.1145/3643991.3644917>
- [12] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of ChatGPT in code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–28, 2025, art. no. 42. [Online]. Available: <https://doi.org/10.1145/3697010>
- [13] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 50 528–50 652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [14] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024, art. no. 220. [Online]. Available: <https://doi.org/10.1145/3695988>